

School of Computer Science and Artificial Intelligence

Lab Assignment # 14.5

Program : B. Tech (CSE)
Specialization : CSE
Course Title : AI Assisted Coding
Course Code : 23CS002PC304
Semester : II
Academic Session : 2025-2026
Name of Student A. Vamshi Krishna
Enrollment :2403A51L47
Batch No : 52

Prompt:

```
Create a responsive Image Gallery Website using only HTML, CSS, and Bootstrap (do NOT use React or any framework).

Requirements:

1. Structure:
- Provide complete project structure (index.html, css/styles.css, optional js/script.js).
- Code should be beginner-friendly and well organized.

2. Layout:
- Use CSS Grid for the gallery layout.
- Use Bootstrap for container, spacing, and responsive utilities.
- Display images in a clean grid layout.

3. Images:
- Use online images from Unsplash or Picsum (no local images).
- Add proper alt text.

4. Styling (Important):
- Make the design colorful using gradients and modern UI styles.
- Add card shadow, rounded corners, and smooth hover effects.

5. Effects:
- Add hover zoom effect on images.
- Add overlay text appearing on hover.

6. Responsiveness:
- Fully responsive for mobile, tablet, and desktop.
- Use auto-fit + minmax for grid responsiveness.

7. Output Format:
- Provide full working code for each file separately.
- Explain briefly how responsiveness works.

Goal:
The gallery should look modern, colorful, and smooth with clean animations.
```

Code Screenshot:

```
index.html X # styles.css prompt.txt
image-gallery-project > index.html > html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6
7 <title>Colorful Image Gallery</title>
8
9 <!-- Bootstrap -->
10 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
11
12 <link rel="stylesheet" href="css/styles.css">
13 </head>
14
15 <body>
16
17 <!-- Navbar -->
18 <nav class="navbar navbar-expand-lg navbar-dark bg-gradient">
19 <div class="container">
20 <a class="navbar-brand fw-bold">🎨 Color Gallery</a>
21 </div>
22 </nav>
23
24 <!-- Hero -->
25 <div class="hero text-center text-white">
26 <h1 class="fw-bold">Modern Responsive Image Gallery</h1>
27 <p>Bootstrap + CSS Grid + Hover Effects</p>
28 </div>
29
30 <!-- Gallery -->
31 <div class="container py-5">
32
33 <div class="gallery">
34
35 <!-- Picsum random -->
36 <div class="gallery-item">
37 
38 <div class="overlay">Creative Design</div>
39 </div>
40
41 <div class="gallery-item">
42 
43 <div class="overlay">Workspace</div>
44 </div>
```

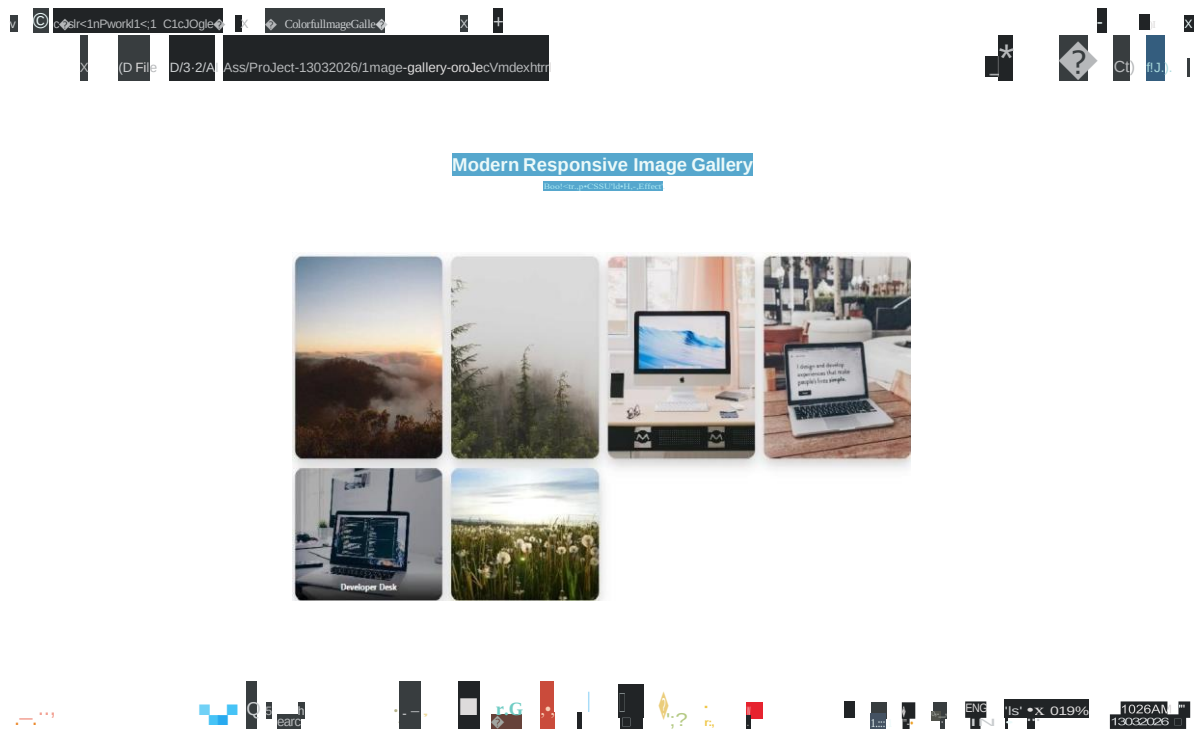
```
index.html  img1.jpg  # styles.css  X  prompt.txt
mage-gallery-project > css > # styles.css > {} @media (max-width:500px)
1  /* ----- Navbar Gradient ----- */
2
3  .bg-gradient {
4  background: linear-gradient(45deg, #ff6a00, #ee0979);
5  }
6
7  /* ----- Hero Section ----- */
8
9  .hero {
10 padding: 60px 20px;
11 background: linear-gradient(120deg, #36d1dc, #5b86e5);
12 }
13
14 /* ----- Gallery Layout ----- */
15
16 .gallery {
17 display: grid;
18 grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
19 gap: 20px;
20 }
21
22 /* ----- Cards ----- */
23
24 .gallery-item {
25 position: relative;
26 overflow: hidden;
27 border-radius: 15px;
28 box-shadow: 0 10px 25px rgba(0,0,0,0.2);
29 }
30
31 /* Images */
32
33 .gallery-item img {
34 width: 100%;
35 height: 100%;
36 object-fit: cover;
37 transition: transform .4s ease, filter .4s ease;
38 }
39
40 /* Hover Effects */
41
42 .gallery-item:hover img {
43 transform: scale(1.1);
44 filter: brightness(70%);
```

```

v image-gallery-project
├── css
├── images
├── js
├── index.html
└── prompt.txt

```

Code Output:



Task 2: Profile Card Generator.

Structure:

```

.....
ro | 1: e-card-we -page
├── css
├── js
├── index.html
└── prompt.bd

```

Prompt:

pmrnp.txt

Create a modern Profile Card web page using only HTML, CSS, and Bootstrap (no React).

Requirements:

1. Structure:

- Provide full project structure (index.html, css/styles.css, optional js/script.js).
- Keep code beginner-friendly and clean.

2. Card Content:

- Show user image, name, role, short bio, and social media links.

3. Layout:

- Use Flexbox to design the profile card layout.
- Center the card vertically and horizontally.

4. Styling:

- Suggest modern UI colors and Google Fonts.
- Use gradients, rounded corners, and soft shadows.

5. 'Effects:

- Add hover animation on card.
- Add hover effects on social icons.

6. Responsiveness:

- Fully responsive for mobile and desktop.

7. Output:

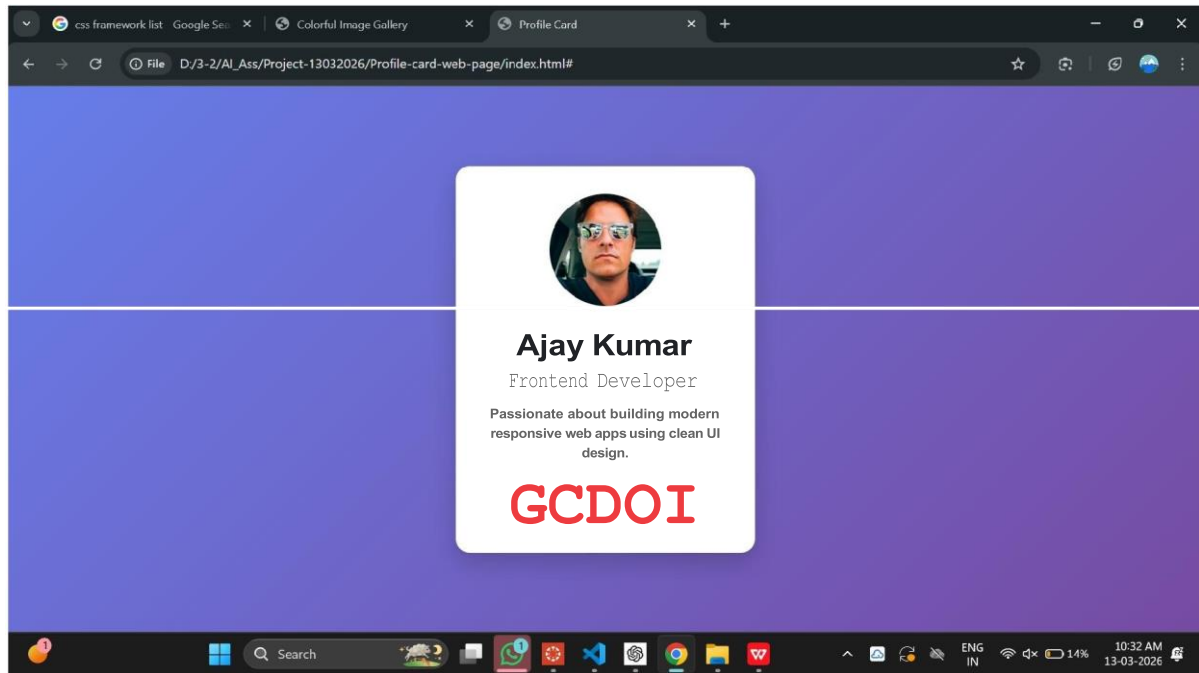
- Provide full working code for each file separately.

Code Screenshot:

```
index.html X # styles.css prompt.txt
Profile-card-web-page > <> index.html <> html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6
7 <title>Profile Card</title>
8
9 <!-- Bootstrap -->
10 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
11
12 <!-- Google Font -->
13 <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;500;700&display=swap" rel="stylesheet">
14
15 <!-- Icons -->
16 <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome@6.5.0/css/all.min.css">
17
18 <link rel="stylesheet" href="css/styles.css">
19 </head>
20
21 <body>
22
23 <div class="main-container">
24
25 <div class="profile-card">
26
27 
28
29 <h2>Ajay Kumar</h2>
30 <h3>Frontend Developer</h3>
31
32 <p class="bio">
33 Passionate about building modern responsive web apps using clean UI design.
34 </p>
35
36 <div class="social-icons">
37
38 <a href="#"><i class="fab fa-linkedin"></i></a>
39 <a href="#"><i class="fab fa-github"></i></a>
40 <a href="#"><i class="fab fa-twitter"></i></a>
41 <a href="#"><i class="fab fa-instagram"></i></a>
42
43 </div>
44
```

```
index.html # styles.css X prompt.txt
Profile-card-web-page > css > # styles.css > .social-icons a:hover
1  /* ----- Global ----- */
2
3  body {
4    margin: 0;
5    font-family: 'Poppins', sans-serif;
6
7    background: linear-gradient(135deg, #667eea, #764ba2);
8    height: 100vh;
9  }
10
11 /* ----- Center Layout (Flexbox) ----- */
12
13 .main-container {
14   display: flex;
15   justify-content: center;
16   align-items: center;
17   height: 100%;
18 }
19
20 /* ----- Card ----- */
21
22 .profile-card {
23   background: white;
24   padding: 30px;
25   width: 320px;
26   border-radius: 15px;
27   text-align: center;
28
29   box-shadow: 0 10px 25px rgba(0,0,0,0.2);
30   transition: .4s;
31 }
32
33 /* Hover Animation */
34
35 .profile-card:hover {
36   transform: translateY(-10px);
37   box-shadow: 0 15px 35px rgba(0,0,0,0.3);
38 }
39
40 /* ----- Image ----- */
41
42 .profile-img {
43   width: 120px;
44   height: 120px;
```

Output Screenshot:



Task 3: Develop a dynamic shopping cart system.

Prompt:

```
prompt.txt
Create a dynamic shopping cart system using HTML, CSS, Bootstrap, and JavaScript (no React).

Requirements:

1. Structure:
- Provide full project structure (index.html, css/styles.css, js/script.js).
- Code should be beginner-friendly and clean.

2. Features:
- Display product cards with image, name, price, and Add to Cart button.
- Allow users to add, remove, and update product quantities.

3. Logic (Important):
- Manage cart state using JavaScript objects.
- Update total price dynamically in real time.
- Persist cart data using localStorage.

4. UI:
- Use Bootstrap for layout and responsiveness.
- Add hover effects and clean modern styling.

5. Output:
- Provide full working code for each file separately.

23
```

Code Screenshot:

```

index.html X # styles.css prompt.txt
Profile-card-web-page > index.html > html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6
7 <title>Shopping Cart</title>
8
9 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
10 <link rel="stylesheet" href="css/styles.css">
11 </head>
12
13 <body>
14
15 <div class="container py-5">
16
17 <h2 class="text-center mb-4">Dynamic Shopping Cart</h2>
18
19 <div class="row" id="product-list"></div>
20
21 <hr>
22
23 <h4>Cart</h4>
24 <div id="cart-items"></div>
25
26 <h3 class="mt-3">Total: ₹<span id="total">0</span></h3>
27
28 </div>
29
30 <script src="js/script.js"></script>
31 </body>
32 </html>

```

```

index.html # styles.css X prompt.txt
Profile-card-web-page > css > # styles.css > body
1 /* ----- Global ----- */
2
3 body {
4   margin: 0;
5   font-family: 'Poppins', sans-serif;
6
7   background: linear-gradient(135deg, #667eea, #764ba2);
8   height: 100vh;
9 }
10
11 /* ----- Center Layout (Flexbox) ----- */
12
13 .main-container {
14   display: flex;
15   justify-content: center;
16   align-items: center;
17   height: 100%;
18 }
19
20 /* ----- Card ----- */
21
22 .profile-card {
23   background: white;
24   padding: 30px;
25   width: 320px;
26   border-radius: 15px;
27   text-align: center;
28
29   box-shadow: 0 10px 25px rgba(0,0,0,0.2);
30   transition: .4s;
31 }
32
33 /* Hover Animation */
34
35 .profile-card:hover {
36   transform: translateY(-10px);
37   box-shadow: 0 15px 35px rgba(0,0,0,0.3);
38 }
39
40 /* ----- Image ----- */
41
42 .profile-img {
43   width: 120px;
44   height: 120px;

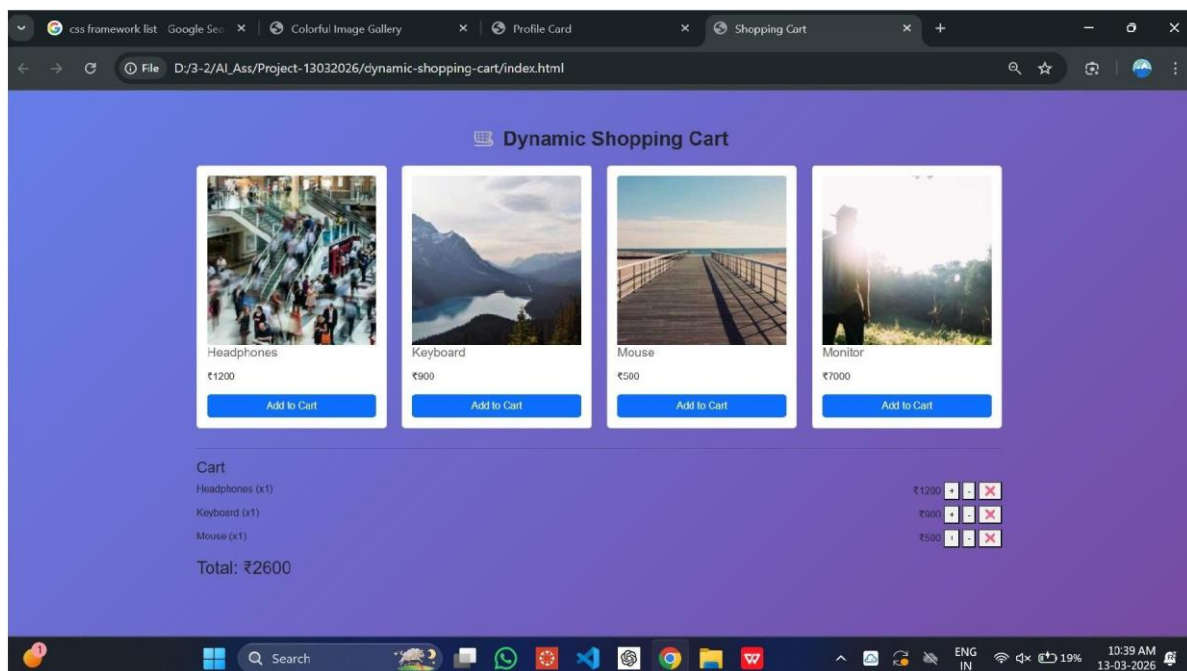
```

```

dynamic-shopping-cart > js > JS script.js > ...
16 function renderProducts(){
19 products.forEach(p=>{
22 <div class="card p-3 mb-3">
23 
24 <h5>${p.name}</h5>
25 <p>₹${p.price}</p>
26 <button class="btn btn-primary" onclick="addToCart(${p.id})">Add to Cart</button>
27 </div>
28 </div>
29 ~;
30 ~});
31 ~}
32
33 // ----- Add to Cart -----
34
35 function addToCart(id){
36 cart[id]=cart[id]?cart[id]+1:1;
37 updateCart();
38 }
39
40 // ----- Update Cart UI -----
41
42 function updateCart(){
43 localStorage.setItem("cart",JSON.stringify(cart));
44
45 const cartDiv=document.getElementById("cart-items");
46 cartDiv.innerHTML="";
47 let total=0;
48
49 for(let id in cart){
50 let product=products.find(p=>p.id==id);
51 let qty=cart[id];
52 let subtotal=product.price*qty;
53
54 total+=subtotal;
55
56 cartDiv.innerHTML+=`
57 <div class="d-flex justify-content-between mb-2">
58 <span>${product.name} (x${qty})</span>
59 <span>
60 ₹${subtotal}
61 <button onclick="changeQty(${id},1)">+</button>
62 <button onclick="changeQty(${id},-1)">-</button>
63 <button onclick="removeItem(${id})">✖</button>

```

Output Screenshot:



Structure:

```

v dynamic-shopping-cart
  v CSS
    .ti styles.css
  v JS
    JS script.js
  • index.html
  ? prompt.txt
  
```

Task 4: Role-Based Access Control Application.

Prompt:

```

1 Create a Role-Based Access Control (RBAC) web application using HTML, CSS, Bootstrap, and JavaScript (no React).

Requirements:

1. Structure:
   Provide full project structure (index.html, css/styles.css, js/script.js).
   Keep code clean and beginner friendly.

2. Roles:
   - Implement Admin, Editor, and User roles.

3. Features:
   Login simulation with role selection.
   Conditional UI rendering based on role.
   Restrict actions depending on role permissions.

4. Logic (Important):
   Use JavaScript objects to define role permissions.
   Maintain secure state Transitions.
   Store role state in localStorage.

5. UI:
   Use Bootstrap components.
   - Create scalable reusable UI sections.

6. Output:
   - Provide full working code for each file separately.
  
```

Structure:

```

v Role-based-access-control (RBAC)
  v CSS
    # styles.css
  v JS
    JS script.js
    index.html
  ? prompt.txt
  
```

Code Screenshot:

```
index.html X # styles.css prompt.txt JS script.js
Role-based-access-control (RBAC) > index.html > html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6
7 <title>RBAC App</title>
8
9 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
10 <link rel="stylesheet" href="css/styles.css">
11 </head>
12
13 <body>
14
15 <div class="container py-5">
16
17 <h2 class="text-center mb-4">🔒 Role-Based Access Control</h2>
18
19 <!-- Login Simulation -->
20 <div class="text-center mb-4">
21 <select id="roleSelect" class="form-select w-25 mx-auto">
22 <option value="">Select Role</option>
23 <option value="admin">Admin</option>
24 <option value="editor">Editor</option>
25 <option value="user">User</option>
26 </select>
27
28 <button class="btn btn-primary mt-2" onclick="login()">Login</button>
29 <button class="btn btn-danger mt-2" onclick="logout()">Logout</button>
30 </div>
31
32 <hr>
33
34 <!-- Dashboard -->
35 <div id="dashboard"></div>
36
37 </div>
38
39 <script src="js/script.js"></script>
40 </body>
41 </html>
```

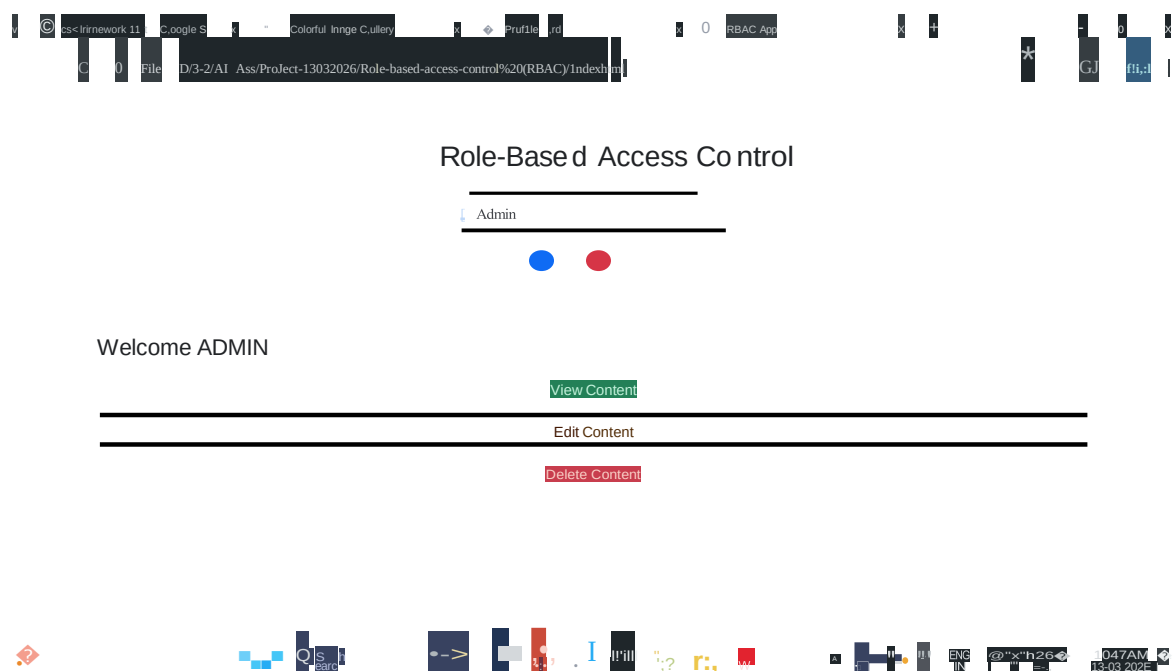
```
index.html # styles.css X prompt.txt JS script.js
Role-based-access-control (RBAC) > css > # styles.css > .cardhover
1 .card{
2   transition:.3s;
3 }
4
5 .card:hover{
6   transform: translateY(-5px);
7 }
```

```

Role-based-access-control (RBAC) > js > JS script.js > ...
1 // ----- Role Permissions -----
2
3 const roles = {
4   admin: ["view", "edit", "delete"],
5   editor: ["view", "edit"],
6   user: ["view"]
7 };
8
9 // ----- State -----
10
11 let currentRole = localStorage.getItem("role") || null;
12
13 // ----- Login -----
14
15 function login(){
16   const role=document.getElementById("roleSelect").value;
17   if(!role) return alert("Select role");
18
19   currentRole=role;
20   localStorage.setItem("role",role);
21   renderUI();
22 }
23
24 // ----- Logout -----
25
26 function logout(){
27   localStorage.removeItem("role");
28   currentRole=null;
29   renderUI();
30 }
31
32 // ----- Conditional Rendering -----
33
34 function renderUI(){
35   const dash=document.getElementById("dashboard");
36
37   if(!currentRole){
38     dash.innerHTML="<p class='text-center'>Please login</p>";
39     return;
40   }
41
42   let perms=roles[currentRole];
43
44   dash.innerHTML="

```

Code Output:



Task 5: Multi-Language Website

Prompt:

```

prompt<
Create a multilingual website using HTML, CSS, Bootstrap, and JavaScript (no React).

Requirements:

1. Structure:
- Provide full project structure (index.html, css/styles.css, js/script.js).
- Keep code beginner-friendly and clean.

2. Features:
- Dynamically switch languages (English, Hindi, Arabic).
- Use structured translation files (JavaScript objects).

3. Logic (Important):
- Persist selected language using localStorage.
- Automatically apply saved language on reload.

4. Layout:
- Support both LTR and RTL layouts.
- Switch direction dynamically when Arabic is selected.

5. Accessibility:
- Use semantic HTML.
- Add aria-labels where needed.

6. Output:
- Provide full working code for each file separately -l
26

```

Structure:

```

PROJECT-13032026 Co
v Project
v CSS
# styles.css
v JS
JS scriptJs
index.html
prompt.txt

```

Code Screenshot:

```

index.html X # styles.css prompt.txt JS script.js
Role-based-access-control (RBAC) > index.html > html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6
7 <title>Multi Language Website</title>
8
9 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
10 <link rel="stylesheet" href="css/styles.css">
11 </head>
12
13 <body>
14
15 <div class="container py-5">
16
17 <h2 id="title">Multi Language Website</h2>
18
19 <!-- Language Switch -->
20 <select id="langSelect" class="form-select w-25 mb-4" aria-label="Language Switcher">
21 <option value="en">English</option>
22 <option value="hi">Hindi</option>
23 <option value="ar">Arabic</option>
24 </select>
25
26 <p id="desc"></p>
27
28 <button class="btn btn-primary" id="btn"></button>
29
30 </div>
31
32 <script src="js/script.js"></script>
33 </body>
34 </html>

```

Project > css > styles.css > body

bodi(1

transition: 3s;

3 ?

```
Project > js > JS scriptjs > ...
1 // ----- Translations -----
2
3 const translations = {
4
5   en:{
6     title:"Multi Language Website",
7     desc:"This website supports multiple languages dynamically.",
8     btn:"Click Me"
9   },
10
11   hi:{
12     title:"बहुभाषी वेबसाइट",
13     desc:"यह वेबसाइट कई भाषाओं का समर्थन करती है।",
14     btn:"क्लिक करें"
15   },
16
17   ar:{
18     title:"موقع متعدد اللغات",
19     desc:"هذا الموقع يدعم لغات متعددة.",
20     btn:"اضغط هنا"
21   }
22
23 };
24
25 // ----- Change Language -----
26
27 function setLanguage(lang){
28
29   localStorage.setItem("lang",lang);
30
31   document.getElementById("title").innerText=translations[lang].title;
32   document.getElementById("desc").innerText=translations[lang].desc;
33   document.getElementById("btn").innerText=translations[lang].btn;
34
35   // RTL Support
36   document.body.dir=(lang=="ar")?"rtl":"ltr";
37 }
38
39 // ----- Event -----
40
41 document.getElementById("langSelect").addEventListener("change",(e)->{
42   setLanguage(e.target.value);
43 });
44
```

Code Output:



बहुभाषी वेबसाइट

Hindi

'lo??'fl" 1Jif <1>1 Wl'fi 'l>«fi%1

Iwuii•

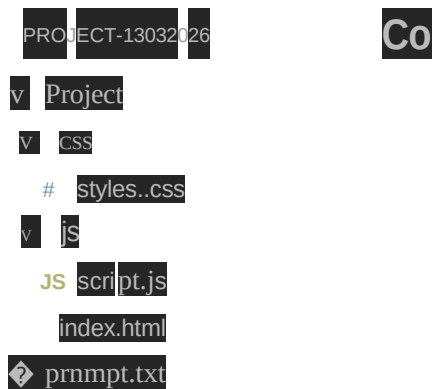
Task 5: Online Examination System

Prompt:

prompt.txt

1. Analyze student performance and give suggestions.
2. Detect suspicious behavior (tab switching/inactivity).
3. Improve ux with reminders.
4. Simplify questions for accessibility.

Structure:



Code Screenshot:

```

index.html X # styles.css prompt.txt JS script.js
Project > index.html > html
 2 <html lang="en">
10 <body>
12 <div class="container">
20 <form id="examForm">
22 <div class="question">
25 <label><input type="radio" name="q1" value="O(n)"> O(n)</label>
26 <label><input type="radio" name="q1" value="O(log n)"> O(log n)</label>
27 <label><input type="radio" name="q1" value="O(n log n)"> O(n log n)</label>
28 </div>
29 </div>
30
31 <div class="question">
32 <p>2. Which data structure uses FIFO?</p>
33
34 <label><input type="radio" name="q2" value="Stack"> Stack</label>
35 <label><input type="radio" name="q2" value="Queue"> Queue</label>
36 <label><input type="radio" name="q2" value="Tree"> Tree</label>
37
38 </div>
39
40 <button type="submit">Submit Exam</button>
41
42 </form>
43
44 <div id="result"></div>
45
46 </div>
47
48 <script src="js/script.js"></script>
49 </body>
50 </html>

```

```
<> index.html # styles.css X ≡ prompt.txt JS script.js
Project > css > # styles.css > #result
1  body {
2  font-family: Arial, sans-serif;
3  background: #f4f6f8;
4  margin: 0;
5  padding: 0;
6  }
7
8  .container {
9  width: 60%;
10 margin: auto;
11 margin-top: 40px;
12 background: white;
13 padding: 20px;
14 border-radius: 10px;
15 box-shadow: 0px 0px 10px rgba(0,0,0,0.1);
16 }
17
18 h1 {
19 text-align: center;
20 }
21
22 .top-bar {
23 display: flex;
24 justify-content: space-between;
25 margin-bottom: 20px;
26 font-weight: bold;
27 }
28
29 .question {
30 margin-bottom: 20px;
31 }
32
33 button {
34 padding: 10px 20px;
35 border: none;
36 background: #007bff;
```

```

index.html # styles.css prompt.txt JS script.js X
Project > js > JS script.js > ...
67 function autoSubmit() {
68   alert("Time is up! Auto submitting.");
69   submitExam();
70 }
71
72 // ===== AI FEATURES =====
73
74 // AI UX Improvement (simple simulation)
75 function runAIUX(score) {
76
77   let aiMessage = "";
78
79   if (score === 2)
80     aiMessage = "AI Suggestion: Excellent performance!";
81   else if (score === 1)
82     aiMessage = "AI Suggestion: Revise DSA basics.";
83   else
84     aiMessage = "AI Suggestion: Focus on core concepts.";
85
86   resultDiv.innerHTML += `<br>${aiMessage}`;
87 }
88
89 // ===== TAB SWITCH MONITOR =====
90
91 document.addEventListener("visibilitychange", () => {
92
93   if (document.hidden) {
94     alert("Warning: Tab switching detected!");
95   }
96
97 });

```

Code Output:

Online Examination

Time Left: 9:40

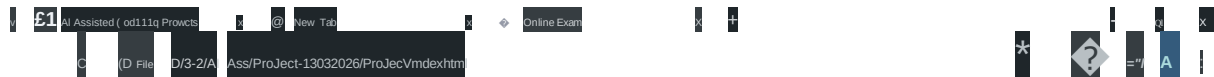
1. What is Big-O of binary search?

☐ $O(n)$ ☒ $O(\log n)$ ☐ $O(n \log n)$

2. Which data structure uses FIFO?

☐ Stack ☒ Queue ☐ Tree

[▶IMHSE5](#)

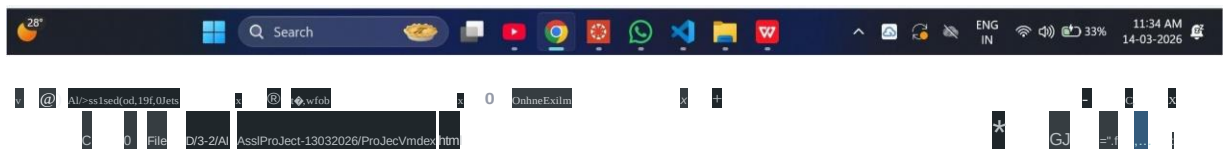


Online Examination

Time Left: 9:30

Your Score: 212

AI Suggestion: Excellent performance!



Online Examination

Time Left: 9:58

Your Score: 0/2

AI Suggestion: Focus on core concepts.

